UNIVERSIDAD TÉCNICA DE AMBATO
FORMULARIO DE GUÍAS PRÁCTICAS DE APLICACIÓN Y EXPERIMENTACIÓN DE LOS APRENDIZAJES

FACULTAD: INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL		**CARRERA:** Software
NIVEL: PRIMERO	**UOC:** Unidad Básica	**ASIGNATURA:** ALGORITMOS Y LOGICA DE PROGRAMACION
CICLO ACADÉMICO: ENERO 2026 - JULIO 2026		
NOMBRES: López Steven Domínguez Daniel Rivadeneyra Matías		

I. TEMA:

Resolución de ejercicios de la prueba de unidad

II. OBJETIVO:

Conocer los lenguajes de programación de la actualidad y sus antecesores. Y sus aplicaciones practicas

III. INSTRUCCIONES:

- Realizar los ejercicios donde cada ejercicio tiene que tener:

- Análisis del problema
- Diseño
- Algoritmos
- Pseudocódigo (pSeInt)
- Diagrama de flujo (pSeInt)
- Código en C++ (VS Code)
- 3 casos de prueba explicados

IV. LISTADO DE EQUIPOS, MATERIALES Y RECURSOS:

- Inteligencia artificial
- Computador o laptop
- Plataforma educativa de la UTA.
- Git Hub

V. ACTIVIDADES POR DESARROLLAR:

Desarrollar la tarea en el formato

Grupo J. Restaurante universitario con pedidos y tiempos de preparación
Contexto del caso: Manejar platos, pedidos, ingredientes, tiempos de preparación, retrasos y facturación.
Clases mínimas sugeridas: Plato y Pedido
Requerimientos funcionales obligatorios

- Registrar platos con código, categoría, costo, precio, tiempo estimado y disponibilidad.
- Registrar pedidos con cliente, mesa, hora, prioridad y detalle de platos.
- Calcular subtotal, descuento, recargo por urgencia e IVA.
- Buscar por cliente y código, ordenar manualmente por mayor demanda y tiempo de espera.
- Actualizar stock básico de ingredientes y emitir alertas por retraso.

Integración obligatoria con Ahorcado: Construir un Ahorcado con platos, ingredientes, procesos, pedidos y tiempos.
Fragmento defectuoso para depurar y justificar

```
void agregarTiempo(int &tiempo, int extra){
   tiempo =+ extra;
   if(tiempo > 60)
      tiempo == 60;
}
```

El estudiante debe corregir el fragmento, justificar los errores encontrados y demostrar la corrección con una prueba mínima.

1. Análisis

Se creará un programa en C++ que permita al usuario registrar los platos que se disponen en un restaurante, además de registrar los pedidos de clientes tomando en cuenta la disponibilidad, tiempo de preparación y urgencia del pedido. Esto permite al usuario llevar un orden de los platos, así como su disponibilidad, y conocer el estado de los pedidos.

- Actores: Cliente, sistema, cocina
- Variables clave:

Plato: código, categoría, costo, precio, tiempo, disponibilidad
Pedido: cliente, mesa, hora, prioridad, lista de platos

- Casos críticos:

Pedido urgente: recargo
Tiempo > 60: límite
Stock insuficiente: alerta
Letras inválidas en ahorcado

2. Diseño

- Clases:

Plato
Pedido

- Estructuras:

vector<Plato>
vector<Pedido>
map<string,int> (stock ingredientes)

- Variables:

<u>Entrada</u>
Int código;
String nombre, categoría;
Float costo, precio;
Int tiempoPreparacion;
Boolean disponible;
Int demanda;
string cliente;
int mesa;
string hora;

int prioridad;
string palabra;
int opcion;
Proceso
Se solicita los datos necesarios al usuario para su ejecución correcta, por ende, una vez obtenida toda esta información, el programa procederá a realizar la evaluación y los cálculos pertinentes para llegar al resultado esperado y estructurado de cómo se llegó a plantear desde un inicio
Salida
Al usuario se le mostraran las siguientes opciones una vez que haya finalizada la ejecución previa del código: Registrar Plato, Crear Pedido, Buscar Cliente, Ordenar por Demanda, Ordenar por Tiempo, Jugar Ahorcado,

3. Depuración

Tomando en cuenta el siguiente fragmento:

```
void agregarTiempo(int &tiempo, int extra){
   tiempo =+ extra;
   if(tiempo > 60)
      tiempo == 60;
}
```

Se puede observar que existen 2 errores de sintaxis:

1. El acumulador no debe ser =+, sino +=.
2. No se debe usar ==, sino =. Esto debido a que se está asignando, no comparando.

4. Algoritmo

1. Inicializar variables:
   - stockIngredientes ← 10
   - partidasGanadas ← 0
   - partidasPerdidas ← 0
   - totalPlatos ← 0
   - totalPedidos ← 0
2. Declarar arreglos para:
   - Platos: código, nombre, categoría, costo, precio, tiempo, demanda.
   - Pedidos: cliente, mesa, prioridad, subtotal, total.
3. Repetir hasta que el usuario elija salir (op = 0):
   - Mostrar menú:

1. Registrar Plato
2. Crear Pedido
3. Buscar Cliente
4. Ordenar por Demanda
5. Ordenar por Tiempo
6. Jugar Ahorcado
0. Salir

   - Leer opción op
4. Incrementar totalPlatos
5. Ingresar datos del plato:
   - Código, nombre, categoría, costo, precio, tiempo
6. Inicializar demanda = 0

7. Incrementar totalPedidos
8. Ingresar:
    - Cliente, mesa, prioridad
9. Inicializar subtotal = 0
10. Repetir:

- Mostrar lista de platos
- Leer opción (op2)
- Si es válida:
    - Sumar precio al subtotal
    - Incrementar demanda del plato
- Hasta que op2 = -1

11. Calcular:

- Descuento = 10% del subtotal
- Recargo = 5% si prioridad = 3
- IVA = 12%
- Total = subtotal - descuento + recargo + IVA

12. Mostrar subtotal y total
13. Reducir stockIngredientes
14. Si stock ≤ 2 → mostrar alerta
15. Leer nombre a buscar
16. Recorrer pedidos:

- Si coincide:
    - Mostrar cliente, subtotal y total

17. Aplicar ordenamiento (tipo burbuja):

- Ordenar de mayor a menor demanda
- Intercambiar también nombres

18. Mostrar mensaje de ordenamiento
19. Aplicar ordenamiento:

- Ordenar de menor a mayor tiempo
- Intercambiar también nombres

20. Mostrar mensaje
21. Mostrar submenú:

1. 1 vs 1
2. Contra CPU
3. Resultados
4. Instrucciones
5. Volver

22. Ingresar palabra secreta
23. Crear progreso con guiones
24. Intentos = 6
25. Repetir:

- Mostrar progreso e intentos
- Leer letra
- Verificar si existe en palabra
- Si no acierta → reducir intentos
- Hasta ganar o perder

26. Actualizar:

- Ganadas o perdidas

27. Elegir dificultad:

- Fácil → "pizza"
- Medio → "encebollado"
- Difícil → "yahuarlocro"

28. Mostrar partidas ganadas y perdidas
29. Mostrar reglas del juego
30. Terminar cuando op = 0

5. Pseudocódigo

```
Proceso SistemaRestauranteYAhorcado

Definir stockIngredientes Como Entero
stockIngredientes <- 10

Definir partidasGanadas, partidasPerdidas Como Entero
partidasGanadas <- 0
partidasPerdidas <- 0

Dimension codigo[100], tiempoPreparacion[100], demanda[100]
Dimension nombre[100], categoria[100]
Dimension costo[100], precio[100]
Definir totalPlatos Como Entero
totalPlatos <- 0

Dimension clientePedido[100], mesaPedido[100], prioridadPedido[100]
Dimension subtotalPedido[100], totalPedido[100]
Definir totalPedidos Como Entero
totalPedidos <- 0

Definir op Como Entero

Repetir

Escribir "1. Registrar Plato"
Escribir "2. Crear Pedido"
Escribir "3. Buscar Cliente"
```

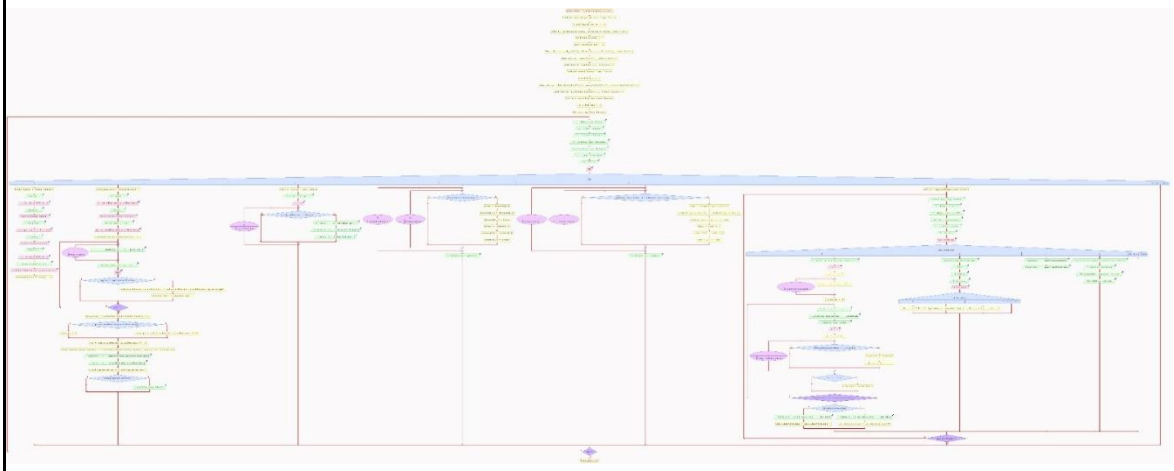
```
Escribir "4. Ordenar por Demanda"
Escribir "5. Ordenar por Tiempo"
Escribir "6. Jugar Ahorcado"
Escribir "0. Salir"
Leer op

Segun op Hacer

    1:
    totalPlatos <- totalPlatos + 1

    Escribir "Codigo: "
    Leer codigo[totalPlatos]

    Escribir "Nombre: "
    Leer nombre[totalPlatos]

    Escribir "Categoria: "
    Leer categoria[totalPlatos]
        Escribir "Costo: "
        Leer costo[totalPlatos]

        Escribir "Precio: "
        Leer precio[totalPlatos]

        Escribir "Tiempo preparacion: "
        Leer tiempoPreparacion[totalPlatos]

        demanda[totalPlatos] <- 0

    2:
        totalPedidos <- totalPedidos + 1

        Escribir "Cliente: "
        Leer clientePedido[totalPedidos]

        Escribir "Mesa: "
        Leer mesaPedido[totalPedidos]

        Escribir "Prioridad (1-3): "
        Leer prioridadPedido[totalPedidos]

        subtotalPedido[totalPedidos] <- 0

        Repetir

        Para i <- 1 Hasta totalPlatos
        Escribir i,". ",nombre[i]," $",precio[i]
        FinPara

        Escribir "Seleccione (-1 salir): "
        Leer op2

        Si op2 > 0 Y op2 <= totalPlatos Entonces
        subtotalPedido[totalPedidos] <- subtotalPedido[totalPedidos] + precio[op2]
        demanda[op2] <- demanda[op2] + 1
                FinSi

        Hasta Que op2 = -1
```

```
descuento <- subtotalPedido[totalPedidos] * 0.1

Si prioridadPedido[totalPedidos] = 3 Entonces
recargo <- subtotalPedido[totalPedidos] * 0.05
        SiNo
recargo <- 0
        FinSi

iva <- subtotalPedido[totalPedidos] * 0.12

totalPedido[totalPedidos] <- subtotalPedido[totalPedidos] - descuento + recargo + iva

Escribir "Subtotal: $",subtotalPedido[totalPedidos]
Escribir "Total: $",totalPedido[totalPedidos]

stockIngredientes <- stockIngredientes - 1

Si stockIngredientes <= 2 Entonces
Escribir "ALERTA: Poco stock"
        FinSi

3:
Definir buscar Como Cadena
Escribir "Buscar cliente: "
Leer buscar

Para i <- 1 Hasta totalPedidos
Si clientePedido[i] = buscar Entonces
Escribir "Cliente: ",clientePedido[i]
Escribir "Subtotal: $",subtotalPedido[i]
Escribir "Total: $",totalPedido[i]
                        FinSi
                FinPara

4:
Para i <- 1 Hasta totalPlatos-1
Para j <- i+1 Hasta totalPlatos
Si demanda[i] < demanda[j] Entonces

temp <- demanda[i]
demanda[i] <- demanda[j]
demanda[j] <- temp

temp2 <- nombre[i]
nombre[i] <- nombre[j]
nombre[j] <- temp2

                                FinSi
                        FinPara
                FinPara

Escribir "Ordenado por demanda"
5:
Para i <- 1 Hasta totalPlatos-1
Para j <- i+1 Hasta totalPlatos
Si tiempoPreparacion[i] > tiempoPreparacion[j] Entonces

temp <- tiempoPreparacion[i]
tiempoPreparacion[i] <- tiempoPreparacion[j]
tiempoPreparacion[j] <- temp
```

```
temp2 <- nombre[i]
nombre[i] <- nombre[j]
nombre[j] <- temp2

                                        FinSi
                                FinPara
                        FinPara

Escribir "Ordenado por tiempo"

6:
Definir opcionJuego Como Entero

Repetir

Escribir "===== AHORCADO ====="
Escribir "1. Jugar 1 vs 1"
Escribir "2. Jugar contra CPU"
Escribir "3. Ver resultados"
Escribir "4. Instrucciones"
Escribir "5. Volver"
Leer opcionJuego
Segun opcionJuego Hacer

1:
Escribir "Jugador 1 ingrese la palabra:"
Leer palabra

progreso <- ""
Para i <- 1 Hasta Longitud(palabra)
progreso <- progreso + "_"
                        FinPara

intentos <- 6

Repetir

Escribir "Palabra: ",progreso
Escribir "Intentos restantes: ",intentos

Escribir "Ingrese una letra:"
Leer letra

acierto <- Falso

Para i <- 1 Hasta Longitud(palabra)
Si Subcadena(palabra,i,i) = letra Entonces
progreso <- progreso
acierto <- Verdadero
                                FinSi
                        FinPara

Si acierto = Falso Entonces
intentos <- intentos - 1
                                FinSi

Hasta Que intentos = 0 O progreso = palabra

Si progreso = palabra Entonces
```

```
        Escribir "Ganaste! La palabra era: ",palabra
        partidasGanadas <- partidasGanadas + 1
                                SiNo
        Escribir "Perdiste! La palabra era: ",palabra
        partidasPerdidas <- partidasPerdidas + 1
                                FinSi

        2:
        Escribir "Seleccione dificultad"
        Escribir "1 Facil"
        Escribir "2 Medio"
        Escribir "3 Dificil"
        Leer dificultad

        Segun dificultad Hacer
        1: palabra <- "pizza"
        2: palabra <- "encebollado"
        3: palabra <- "yahuarlocro"
                                        FinSegun

        3:
        Escribir "Ganadas: ",partidasGanadas
        Escribir "Perdidas: ",partidasPerdidas

        4:
        Escribir "Adivina la palabra letra por letra"
        Escribir "Tienes 6 intentos"
        Escribir "Solo puedes ingresar letras"
        Escribir "No repetir letras"

                                FinSegun

        Hasta Que opcionJuego = 5

        FinSegun

    Hasta Que op = 0

    FinProceso
```

6. Diagrama de flujo



7. Código en C++ (VS Code)

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
```

```
class Plato {
private:
   int codigo;
   string nombre;
   string categoria;
   float costo;
   float precio;
   int tiempoPreparacion;
   bool disponible;
   int demanda;
public:
   Plato() {
      codigo = 0;
      nombre = "";
      categoria = "";
      costo = 0;
      precio = 0;
      tiempoPreparacion = 0;
      disponible = true;
      demanda = 0;
   }
   void registrar() {
      cout << "Codigo: "; cin >> codigo;
      cout << "Nombre: "; cin >> nombre;
      cout << "Categoria: "; cin >> categoria;
      cout << "Costo: "; cin >> costo;
      cout << "Precio: "; cin >> precio;
      cout << "Tiempo preparacion: "; cin >> tiempoPreparacion;
      disponible = true;
   }

   void mostrar() {
      cout << codigo << " - " << nombre << " $" << precio << endl;
   }

   int getCodigo() { return codigo; }
   float getPrecio() { return precio; }
   int getTiempo() { return tiempoPreparacion; }

   void aumentarDemanda() { demanda++; }
   int getDemanda() { return demanda; }
};

class Pedido {
private:
   string cliente;
   int mesa;
   string hora;
   int prioridad;
   vector<Plato> platos;

public:
   void registrarPedido(vector<Plato>& lista) {
      cout << "Cliente: "; cin >> cliente;
      cout << "Mesa: "; cin >> mesa;

      cin.ignore();
      cout << "Hora (HH:MM): ";
      getline(cin, hora);
```

```
    cout << "Prioridad (1-3): "; cin >> prioridad;

    int op;
    do {
        cout << "\nMENU\n";
        for (int i = 0; i < lista.size(); i++) {
            cout << i + 1 << ". ";
            lista[i].mostrar();
        }

        cout << "Seleccione (-1 salir): ";
        cin >> op;

        if (op > 0 && op <= lista.size()) {
            platos.push_back(lista[op - 1]);
            lista[op - 1].aumentarDemanda();
        }

    } while (op != -1);
  }

  float subtotalRecursivo(int i) {
  if (i >= platos.size()) return 0;
  return platos[i].getPrecio() + subtotalRecursivo(i + 1);
}

  float total() {
    float sub = subtotalRecursivo(0);
    float descuento = sub * 0.1;
    float recargo = (prioridad == 3) ? sub * 0.05 : 0;
    float iva = sub * 0.12;

    return sub - descuento + recargo + iva;
  }

  string getCliente() { return cliente; }

  void mostrar() {
    cout << "\nCliente: " << cliente;
    cout << "\nSubtotal: $" << subtotalRecursivo(0) << endl;
    cout << "\nTotal: $" << total() << endl;
  }
};

// -------- FUNCIONES EXTRAS --------

void buscarCliente(vector<Pedido> pedidos) {
  string nombre;
  cout << "Buscar cliente: ";
  cin >> nombre;

  for (auto p : pedidos) {
    if (p.getCliente() == nombre) {
        p.mostrar();
    }
  }
}

void ordenarDemanda(vector<Plato>& lista) {
```

```
    sort(lista.begin(), lista.end(), [](Plato a, Plato b) {
        return a.getDemanda() > b.getDemanda();
    });
    cout << "Ordenado por demanda\n";
}

void ordenarTiempo(vector<Plato>& lista) {
    sort(lista.begin(), lista.end(), [](Plato a, Plato b) {
        return a.getTiempo() < b.getTiempo();
    });
    cout << "Ordenado por tiempo\n";
}

// -------- STOCK SIMPLE --------
int stockIngredientes = 10;

void usarStock() {
    stockIngredientes--;
    if (stockIngredientes <= 2) {
        cout << " ALERTA: Poco stock\n";
    }
}
// ---------------- VARIABLES GLOBALES ----------------
int partidasGanadas = 0;
int partidasPerdidas = 0;

// ---------------- FUNCION JUEGO ----------------
void jugar(string palabra) {
    string progreso(palabra.size(), '_');
    vector<char> usadas;
    int intentos = 6;

    while (intentos > 0 && progreso != palabra) {
        cout << "\nPalabra: " << progreso << endl;
        cout << "Intentos restantes: " << intentos << endl;

        cout << "Letras usadas: ";
        for (char c : usadas) cout << c << " ";
        cout << endl;

        char letra;
        cout << "Ingrese una letra: ";
        cin >> letra;

        letra = tolower(letra);

        // Validar letra
        if (!isalpha(letra)) {
            cout << " Solo letras\n";
            continue;
        }

        // Evitar repetidas
        if (find(usadas.begin(), usadas.end(), letra) != usadas.end()) {
            cout << "Letra repetida\n";
            continue;
        }

        usadas.push_back(letra);
```

```
        bool acierto = false;
        for (int i = 0; i < palabra.size(); i++) {
            if (tolower(palabra[i]) == letra) {
                progreso[i] = palabra[i];
                acierto = true;
            }
        }

        if (!acierto) intentos--;
    }

    if (progreso == palabra) {
        cout << "\n Ganaste! La palabra era: " << palabra << endl;
        partidasGanadas++;
    } else {
        cout << "\nPerdiste! La palabra era: " << palabra << endl;
        partidasPerdidas++;
    }
}

// ---------------- 1 VS 1 ----------------
void modo1vs1() {
    string palabra;
    cout << "Jugador 1 ingrese la palabra: ";
    cin >> palabra;

    // Limpiar pantalla simple
    for (int i = 0; i < 30; i++) cout << endl;

    jugar(palabra);
}

// ---------------- CPU ----------------
void modoCPU() {
    int opcion;
    cout << "\n--- Seleccione dificultad ---\n";
    cout << "1. Facil\n2. Medio\n3. Dificil\n";
    cin >> opcion;

    string palabra;

    switch (opcion) {
        case 1: palabra = "pizza"; break;
        case 2: palabra = "encebollado"; break;
        case 3: palabra = "yahuarlocro"; break;
        default:
            cout << "Opcion invalida\n";
            return;
    }

    jugar(palabra);
}

// ---------------- INSTRUCCIONES ----------------
void instrucciones() {
    cout << "\n--- INSTRUCCIONES ---\n";
    cout << "- Adivina la palabra letra por letra\n";
    cout << "- Tienes 6 intentos\n";
    cout << "- Solo puedes ingresar letras\n";
    cout << "- No repetir letras\n";
```

```
}

// ---------------- RESULTADOS ----------------
void resultados() {
   cout << "\n--- RESULTADOS ---\n";
   cout << "Ganadas: " << partidasGanadas << endl;
   cout << "Perdidas: " << partidasPerdidas << endl;
}

// ---------------- MENU AHORCADO ----------------
void menuAhorcado() {
   int opcion;

   do {
      cout << "\n===== AHORCADO =====\n";
      cout << "1. Jugar 1 vs 1\n";
      cout << "2. Jugar contra CPU\n";
      cout << "3. Ver resultados\n";
      cout << "4. Instrucciones\n";
      cout << "5. Volver al menu principal\n";
      cout << "Seleccione: ";
      cin >> opcion;

      switch (opcion) {
         case 1: modo1vs1(); break;
         case 2: modoCPU(); break;
         case 3: resultados(); break;
         case 4: instrucciones(); break;
         case 5: cout << "Volviendo...\n"; break;
         default: cout << "Opcion invalida\n";
      }

   } while (opcion != 5);
}

// -------- MAIN --------
int main() {
   vector<Plato> platos;
   vector<Pedido> pedidos;

   int op;

   do {
      cout << "\n1. Registrar Plato";
      cout << "\n2. Crear Pedido";
      cout << "\n3. Buscar Cliente";
      cout << "\n4. Ordenar por Demanda";
      cout << "\n5. Ordenar por Tiempo";
      cout << "\n6. Jugar Ahorcado";
      cout << "\n0. Salir\nOpcion: ";
      cin >> op;

      switch (op) {
         case 1: {
            Plato p;
            p.registrar();
            platos.push_back(p);
            break;
         }
```

```
        case 2: {
            Pedido ped;
            ped.registrarPedido(platos);
            ped.mostrar();
            pedidos.push_back(ped);
            usarStock();
            break;
        }

        case 3:
            buscarCliente(pedidos);
            break;

        case 4:
            ordenarDemanda(platos);
            break;

        case 5:
            ordenarTiempo(platos);
            break;
        case 6:
        menuAhorcado();
    }


  } while (op != 0);

  return 0;
}
```

8. Prueba de escritorio

```
1. Registrar Plato
2. Crear Pedido
3. Buscar Cliente
4. Ordenar por Demanda
5. Ordenar por Tiempo
6. Jugar Ahorcado
0. Salir
Opcion: 1
Codigo: 11122
Nombre: Encebollado
Categoria: Almuerzo
Costo: 3
Precio: 4
Tiempo preparacion: 40

1. Registrar Plato
2. Crear Pedido
3. Buscar Cliente
4. Ordenar por Demanda
5. Ordenar por Tiempo
6. Jugar Ahorcado
0. Salir
Opcion: 2
Cliente: Steven
Mesa: 2
Hora (HH:MM): 12:33
Prioridad (1-3): 2

MENU
1. 11122 - Encebollado $4
Seleccione (-1 salir): 1

MENU
1. 11122 - Encebollado $4
Seleccione (-1 salir): -1

Cliente: Steven
Subtotal: $4

Total: $4.08
```

```
1. 11122 - Encebollado $4
Seleccione (-1 salir): -1

Cliente: Steven
Subtotal: $4

Total: $4.08

1. Registrar Plato
2. Crear Pedido
3. Buscar Cliente
4. Ordenar por Demanda
5. Ordenar por Tiempo
6. Jugar Ahorcado
0. Salir
Opcion: 3
Buscar cliente: Jose

1. Registrar Plato
2. Crear Pedido
3. Buscar Cliente
4. Ordenar por Demanda
5. Ordenar por Tiempo
6. Jugar Ahorcado
0. Salir
Opcion: 3
Buscar cliente: Steven

Cliente: Steven
Subtotal: $4

Total: $4.08

1. Registrar Plato
2. Crear Pedido
3. Buscar Cliente
4. Ordenar por Demanda
5. Ordenar por Tiempo
6. Jugar Ahorcado
0. Salir
Opcion:
```

```
Palabra: ence_ollado
Intentos restantes: 4
Letras usadas: a w e c l u o d n
Ingrese una letra: a
Letra repetida

Palabra: ence_ollado
Intentos restantes: 4
Letras usadas: a w e c l u o d n
Ingrese una letra: s

Palabra: ence_ollado
Intentos restantes: 3
Letras usadas: a w e c l u o d n s
Ingrese una letra: d
Letra repetida

Palabra: ence_ollado
Intentos restantes: 3
Letras usadas: a w e c l u o d n s
Ingrese una letra: h

Palabra: ence_ollado
Intentos restantes: 2
Letras usadas: a w e c l u o d n s h
Ingrese una letra: j

Palabra: ence_ollado
Intentos restantes: 1
Letras usadas: a w e c l u o d n s h j
Ingrese una letra: k

Perdiste! La palabra era: encebollado

===== AHORCADO =====
1. Jugar 1 vs 1
2. Jugar contra CPU
3. Ver resultados
4. Instrucciones
```

VI. RESULTADOS OBTENIDOS:

LINK GIT HUB:

https://github.com/StevenLopez-max/Examen-Unidad-2-Grupo-J

VII. HABILIDADES BLANDAS:

Responsabilidad

VIII. CONCLUSIONES:

- No todos los problemas se resuelven de forma directa, algunos requieren analizar varias condiciones al mismo tiempo.
- La práctica constante ayuda a mejorar la lógica y a cometer menos errores en el código.

IX. RECOMENDACIONES:

- Mantener el código ordenado y claro para facilitar su comprensión.
- Antes de programar, realizar siempre un análisis del problema para evitar errores.